

GRAB Prevention

Guarding RAG Against Breaches

A defensive cybersecurity project focused on detecting and preventing prompt injection attacks against Retrieval-Augmented Generation (RAG) systems.

Github:

[GRAB-Prevention/README.md at main · Mikey-Dowsett/GRAB-Prevention](#)

Team:

Project Lead: Doga Eski

Threat Analyst: Derek Ashdown

Defense Engineer: Lukas Viskovic

RAG Systems Developer: Mikey Dowsett

Evaluation & QA: Mike Ding

The Domain:

Security of LLM Applications

Project Scope:

Domain AI/ML Security — Prompt Injection Defense

System RAG-based chatbot with a document knowledge base

Threat Prompt injection leading to unauthorized data extraction

Approach Defensive — detection, mitigation, and evaluation

Timeline ~1 month

Project Overview:

RAG (Retrieval-Augmented Generation) chatbots supplement AI responses with domain-specific documents — internal business data, policy files, knowledge bases, and

more. This power comes with a risk: prompt injection attacks can trick the model into revealing information it was never meant to expose.

GRAB Prevention studies how these attacks work and builds practical defenses to ensure users can only extract information the system is authorized to share — even when the model has access to sensitive source material internally.

Objective:

The objective of this project is to explore and evaluate a prototype AI-based method for detecting suspicious inputs and demonstrating how they can be handled before reaching critical parts of a system.

System Description:

The system is a Retrieval-Augmented Generation (RAG) chatbot that retrieves information from an internal document store and generates responses using a large language model. User queries are processed through a defensive AI layer before interacting with protected data.

Assets Being Protected:

Source documents used to ground RAG responses (business data, policy files, etc.)

The system prompt and model instructions

Access control logic governing what information the model is permitted to surface

Query logs (prevent leakage through observable side channels)

Initial Threat Assumptions:

Attackers are external users with access to the chat interface but no direct access to the document store

Attackers will attempt to extract document content through crafted natural language inputs

The model is assumed to be a commercially available LLM (e.g., GPT-class) with no fine-tuning

Some attacks may be multi-turn (building context across several messages before extracting data)